

Expert Systems

Lecture 9: Multilayer Neural Networks

Dr. Mohammed A. Altahrawi

University College of Applied Sciences

February 24, 2026

Outlines

- 1 Neural Network Definition
- 2 Back-Propagation
- 3 Back-Propagation Training Steps
- 4 Example
- 5 Sum of Squared Errors (SSE)
- 6 Decision Boundaries
- 7 Backpropagation pros and cons

Outlines

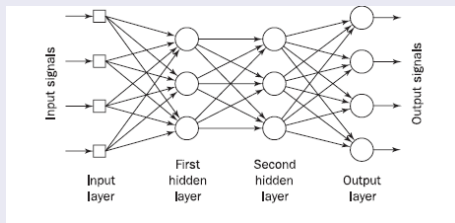
- 1 Neural Network Definition
- 2 Back-Propagation
- 3 Back-Propagation Training Steps
- 4 Example
- 5 Sum of Squared Errors (SSE)
- 6 Decision Boundaries
- 7 Backpropagation pros and cons

Multilayer Neural Networks

Overview

A multilayer perceptron (MLP) is a feedforward neural network with one or more hidden layers.

- Input layer: receives external signals
- Hidden layers: detect features in input patterns
- Output layer: produces the final output pattern



Why Hidden Layers?

Function of Each Layer

- Input layer redistributes signals to the hidden layer (usually no computation)
- Hidden layer neurons detect features; weights represent these features
- Output layer uses hidden layer features to determine final output

Capability

- One hidden layer: can approximate any continuous function
- Two hidden layers: can represent discontinuous functions

Hidden Layer Name

The “hidden” layer hides its desired output; its target output is not directly observable.

Layers and Neurons

- Practical networks usually have 3 layers (input, hidden, output)
- Fully connected: each neuron connects to every neuron in the next layer
- Neuron output:

$$X = \sum_{i=1}^n x_i w_i - \theta$$

$$Y_{\text{sigmoid}} = \frac{1}{1 + e^{-X}}$$

Learning Algorithm

- Back-propagation is the most popular method
- Forward pass: compute output for input patterns
- Backward pass: propagate error from output to hidden layers and update weights

Outlines

- 1 Neural Network Definition
- 2 Back-Propagation**
- 3 Back-Propagation Training Steps
- 4 Example
- 5 Sum of Squared Errors (SSE)
- 6 Decision Boundaries
- 7 Backpropagation pros and cons

Back-Propagation: Output Layer

Error Signal

For output neuron k at iteration p :

$$e_k(p) = y_{d,k}(p) - y_k(p)$$

Weight Update

$$w_{jk}(p+1) = w_{jk}(p) + \Delta w_{jk}(p)$$

$$\Delta w_{jk}(p) = \alpha y_j(p) \delta_k(p)$$

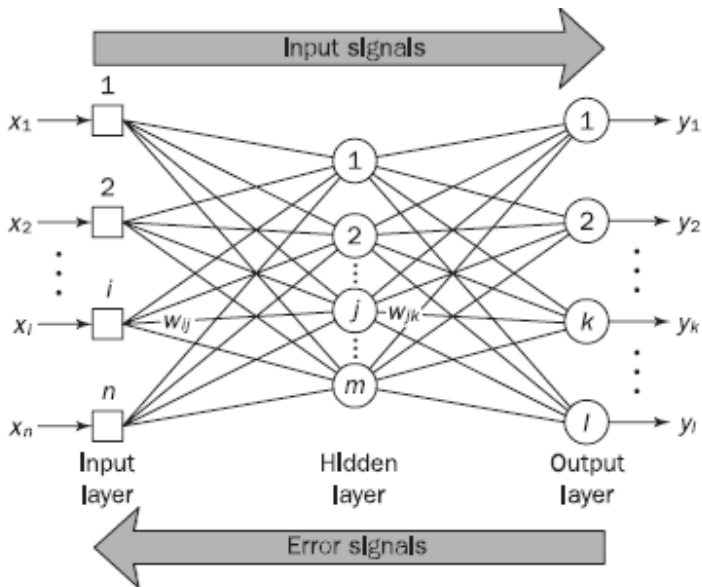
where

$$\delta_k(p) = y_k(p) [1 - y_k(p)] e_k(p)$$

Where

$\Delta w_{jk}(p)$ is the weight correction. $\delta_k(p)$ is the error gradient at neuron k in the output layer at iteration p .

Three-layer back-propagation



What is the Error Gradient?

Definition

The error gradient is determined as the derivative of the activation function multiplied by the error at the neuron output.

For Output Neuron k

$$\delta_k(p) = \frac{\partial y_k(p)}{\partial X_k(p)} e_k(p) \quad (6.13)$$

- $y_k(p)$: output of neuron k at iteration p
- $X_k(p)$: net weighted input to neuron k
- $e_k(p)$: output error

Sigmoid Activation Case

Sigmoid Function

$$y_k(p) = \frac{1}{1 + \exp[-X_k(p)]} \quad (1)$$

Substituting into (6.13)

$$\delta_k(p) = \frac{\partial}{\partial X_k(p)} \left\{ \frac{1}{1 + \exp[-X_k(p)]} \right\} e_k(p) \quad (2)$$

$$= \frac{\exp[-X_k(p)]}{(1 + \exp[-X_k(p)])^2} e_k(p) \quad (3)$$

Important Result

$$\delta_k(p) = y_k(p) [1 - y_k(p)] e_k(p) \quad (6.14)$$

Key Insight

For the sigmoid function:

$$\frac{d}{dX} \sigma(X) = \sigma(X)(1 - \sigma(X))$$

which simplifies backpropagation computations.

Back-Propagation: Hidden Layer

Weight Update for Hidden Neurons

For hidden neuron j :

$$\Delta w_{ij}(p) = \alpha x_i(p) \delta_j(p)$$

$$\delta_j(p) = y_j(p) [1 - y_j(p)] \sum_{k=1}^l \delta_k(p) w_{jk}(p)$$

- l : number of neurons in output layer
- $x_i(p)$: input to neuron j

Hidden Neuron Output

$$y_j(p) = \frac{1}{1 + e^{-X_j(p)}}, \quad X_j(p) = \sum_{i=1}^n x_i(p) w_{ij}(p) - \theta_j$$

Outlines

- 1 Neural Network Definition
- 2 Back-Propagation
- 3 Back-Propagation Training Steps**
- 4 Example
- 5 Sum of Squared Errors (SSE)
- 6 Decision Boundaries
- 7 Backpropagation pros and cons

Back-Propagation Training Algorithm: Step 1 - Initialization

Weight Initialization

Set all weights and thresholds to random numbers in a small range:

$$\left[-\frac{2.4}{F_i}, +\frac{2.4}{F_i} \right]$$

- F_i : total number of inputs to neuron i
- Initialization is done neuron by neuron

Back-Propagation Training Algorithm: Step 2 - Activation

Forward Pass

Apply inputs $x_1(p), x_2(p), \dots, x_n(p)$ and desired outputs $y_{d,1}(p), y_{d,2}(p), \dots$

Compute hidden layer outputs:

$$y_j(p) = \text{sigmoid} \left(\sum_{i=1}^n x_i(p) w_{ij}(p) - \theta_j \right)$$

- n : number of inputs to neuron j
- Sigmoid ensures output is bounded between 0 and 1 and differentiable

Back-Propagation: Output Layer Computation Cont.:Step 2

Output Neuron Activation

For neuron k in the output layer:

$$y_k(p) = \text{sigmoid} \left(\sum_{j=1}^m x_{jk}(p) w_{jk}(p) - \theta_k \right)$$

- m : number of inputs to neuron k
- Sigmoid ensures output between 0 and 1

Weight training Step-3

Error Gradient and Weight Update

$$\delta_k(p) = y_k(p) [1 - y_k(p)] e_k(p), \quad e_k(p) = y_{d,k}(p) - y_k(p)$$

$$\Delta w_{jk}(p) = \eta y_j(p) \delta_k(p), \quad w_{jk}(p+1) = w_{jk}(p) + \Delta w_{jk}(p)$$

Error Gradient for Hidden Neurons

For hidden neuron j :

$$\delta_j(p) = y_j(p) [1 - y_j(p)] \sum_{k=1}^I \delta_k(p) w_{jk}(p)$$

- I : number of neurons in output layer

Weight Update for Hidden Layer

$$\Delta w_{ij}(p) = \eta x_i(p) \delta_j(p), \quad w_{ij}(p+1) = w_{ij}(p) + \Delta w_{ij}(p)$$

Iteration

Increase iteration p by one, go back to Step 2 and repeat the process until the selected error criterion is satisfied.

Outlines

- 1 Neural Network Definition
- 2 Back-Propagation
- 3 Back-Propagation Training Steps
- 4 Example**
- 5 Sum of Squared Errors (SSE)
- 6 Decision Boundaries
- 7 Backpropagation pros and cons

Example: XOR Network Initialization

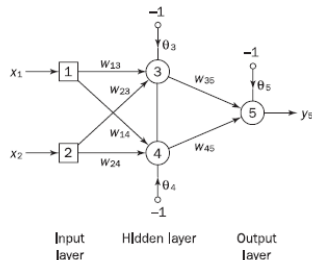
Network Setup

- Input layer: x_1, x_2
- Hidden layer: neurons 3 and 4
- Output layer: neuron 5
- Initial weights and thresholds:

$$w_{13} = 0.5, w_{14} = 0.9, w_{23} = 0.4$$

$$w_{24} = 1.0, w_{35} = -1.2, w_{45} = 1.1$$

$$\theta_3 = 0.8, \theta_4 = -0.1, \theta_5 = 0.3$$



Training Set and Hidden Layer Computation

Given

$$x_1 = 1, \quad x_2 = 1, \quad y_{d,5} = 0$$

The actual outputs of neurons 3 and 4 in the hidden layer are calculated as

$$y_3 = \frac{1}{1 + e^{-(1 \times 0.5 + 1 \times 0.4 - 0.8)}} = 0.5250$$

$$y_4 = \frac{1}{1 + e^{-(1 \times 0.9 + 1 \times 1.0 - (-0.1))}} = 0.8808$$

Output Layer Computation

Now the actual output of neuron 5 in the output layer is determined as:

$$y_5 = \frac{1}{1 + e^{-(0.5250 \times (-1.2) + 0.8808 \times 1.1 - 0.3)}} = 0.5097$$

Error Calculation

$$e = y_{d,5} - y_5 = 0 - 0.5097 = -0.5097$$

Error Gradient for Output Neuron

Error Gradient δ_5 for neuron 5 in the output layer:

$$\delta_5 = y_5(1 - y_5)e$$

$$\delta_5 = 0.5097(1 - 0.5097)(-0.5097)$$

$$\delta_5 = -0.1274$$

Weight Corrections (Output Layer)

Learning Rate

$$\alpha = 0.1$$

Weight Updates

$$\Delta w_{35} = 0.1 \times 0.5250 \times (-0.1274) = -0.0067$$

$$\Delta w_{45} = 0.1 \times 0.8808 \times (-0.1274) = -0.0112$$

$$\Delta \theta_5 = 0.1 \times (-1) \times (-0.1274) = 0.0127$$

calculate the error gradients for neurons 3 and 4 in the hidden layer:

Neuron 3

$$\delta_3 = 0.5250(1 - 0.5250)(-0.1274)(-1.2) = 0.0381$$

Neuron 4

$$\delta_4 = 0.8808(1 - 0.8808)(-0.1274)(1.1) = -0.0147$$

determine the weight corrections

Neuron 3 Updates

$$\Delta w_{13} = 0.1 \times 1 \times 0.0381 = 0.0038$$

$$\Delta w_{23} = 0.1 \times 1 \times 0.0381 = 0.0038$$

$$\Delta \theta_3 = 0.1 \times (-1) \times 0.0381 = -0.0038$$

Neuron 4 Updates

$$\Delta w_{14} = 0.1 \times 1 \times (-0.0147) = -0.0015$$

$$\Delta w_{24} = 0.1 \times 1 \times (-0.0147) = -0.0015$$

$$\Delta \theta_4 = 0.1 \times (-1) \times (-0.0147) = 0.0015$$

Updated Weights and Thresholds

Final Updated Parameters

$$w_{13} = 0.5 + 0.0038 = 0.5038$$

$$w_{23} = 0.4 + 0.0038 = 0.4038$$

$$w_{14} = 0.9 - 0.0015 = 0.8985$$

$$w_{24} = 1.0 - 0.0015 = 0.9985$$

$$w_{35} = -1.2 - 0.0067 = -1.2067$$

$$w_{45} = 1.1 - 0.0112 = 1.0888$$

$$\theta_3 = 0.8 - 0.0038 = 0.7962$$

$$\theta_4 = -0.1 + 0.0015 = -0.0985$$

$$\theta_5 = 0.3 + 0.0127 = 0.3127$$

When is finished

Stopping Condition

Training continues until the sum of squared errors (SSE) is less than 0.001.

Outlines

- 1 Neural Network Definition
- 2 Back-Propagation
- 3 Back-Propagation Training Steps
- 4 Example
- 5 Sum of Squared Errors (SSE)**
- 6 Decision Boundaries
- 7 Backpropagation pros and cons

Why Do We Sum the Squared Errors?

Purpose of SSE

The Sum of Squared Errors (SSE) is a useful indicator of the network's performance.

- Measures total deviation between desired and actual outputs
- Aggregates errors over all training samples
- Always positive (due to squaring)
- Penalizes large errors more heavily

Training Objective

The back-propagation algorithm attempts to minimize the SSE.

Definition

$$SSE = \sum_{p=1}^N \frac{1}{2} \sum_{k=1}^M (y_{d,k}(p) - y_k(p))^2$$

- N = number of training patterns
- M = number of output neurons
- $y_{d,k}(p)$ = desired output
- $y_k(p)$ = actual output

Convergence Criterion

When Does Training Stop?

After a full pass through all training sets (called an **epoch**), the network checks the SSE.

Stopping Condition

If:

$$\text{SSE} < 0.001$$

the network is considered to have converged.

Learning Curve

Learning Curve Concept

A learning curve plots:

SSE vs. Number of Epochs

- Shows how fast the network is learning
- Ideally decreases smoothly
- Plateaus indicate slow learning or convergence

Example from XOR Training

It took:

- 224 epochs
- 896 iterations

to train the network for the XOR operation.

SSE Curve for XOR

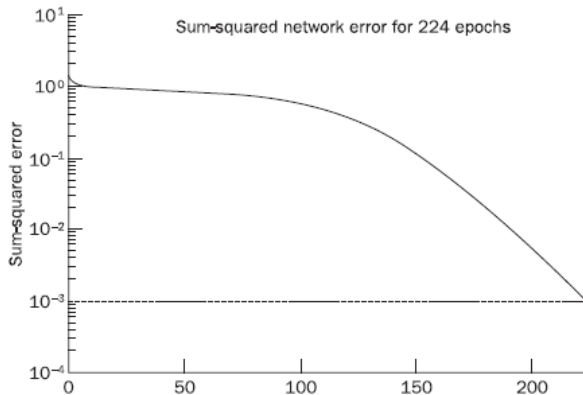


Figure: Learning curve for operation Exclusive-OR

Final Weights and Thresholds

Weights After Convergence

$$w_{13} = 4.7621, \quad w_{23} = 4.7618$$

$$w_{14} = 6.3917, \quad w_{24} = 6.3917$$

$$w_{35} = -10.3788, \quad w_{45} = 9.7691$$

Thresholds (Biases)

$$\theta_3 = 7.3061, \quad \theta_4 = 2.8441, \quad \theta_5 = 4.5589$$

Result

These parameters satisfied the chosen error criterion:

$$\text{SSE} < 0.001$$

Final results

Inputs		Desired output	Actual output	Error	Sum of squared errors
x_1	x_2	y_d	y_s	e	
1	1	0	0.0155	-0.0155	0.0010
0	1	1	0.9849	0.0151	
1	0	1	0.9849	0.0151	
0	0	0	0.0175	-0.0175	

Figure: Final results of three-layer network learning

Do Random Initial Weights Lead to Different Solutions?

Random Initialization

Initial weights and thresholds are set randomly.

- Different initial conditions
- Different optimization paths
- Different final weight values

Important Insight

The network may converge to different numerical solutions, but it still solves the same problem.

Alternative Solution for XOR

Another Converged Set of Parameters

$$w_{13} = -6.3041, \quad w_{14} = -5.7896$$

$$w_{23} = 6.2288, \quad w_{24} = 6.0088$$

$$w_{35} = 9.6657, \quad w_{45} = -9.4242$$

$$\theta_3 = 3.3858, \quad \theta_4 = -2.8976, \quad \theta_5 = -4.4859$$

Conclusion

Different weights, same XOR behavior.

Outlines

- 1 Neural Network Definition
- 2 Back-Propagation
- 3 Back-Propagation Training Steps
- 4 Example
- 5 Sum of Squared Errors (SSE)
- 6 Decision Boundaries**
- 7 Backpropagation pros and cons

Decision Boundaries for XOR

Challenge

Decision boundaries of sigmoid neurons are nonlinear and difficult to visualize directly.

Simplification

We can approximate each neuron using:

McCulloch–Pitts model (sign function)

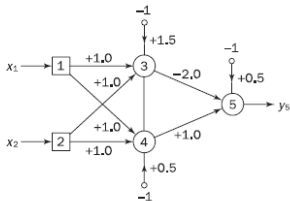


Figure: Network represented by McCulloch–Pitts model

How the Multilayer Network Solves XOR

Hidden Layer

Neurons 3 and 4 construct linear decision boundaries:

$$w_{13}x_1 + w_{23}x_2 - \theta_3 = 0$$

$$w_{14}x_1 + w_{24}x_2 - \theta_4 = 0$$

Output Layer

Neuron 5 forms a linear combination of the hidden outputs.

Key Result

The combination of two linear boundaries creates a nonlinear separation, allowing the network to solve the XOR problem.

Decision boundary

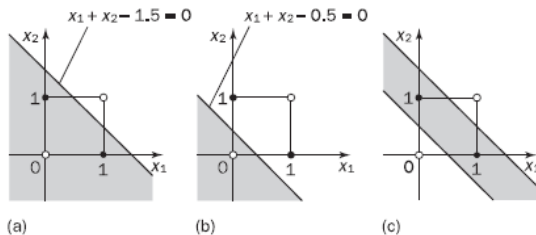


Figure: (a) Decision boundary constructed by hidden neuron 3 of the network in Figure 6.12; (b) decision boundary constructed by hidden neuron 4; (c) decision boundaries constructed by the complete three-layer network

Outlines

- 1 Neural Network Definition
- 2 Back-Propagation
- 3 Back-Propagation Training Steps
- 4 Example
- 5 Sum of Squared Errors (SSE)
- 6 Decision Boundaries
- 7 Backpropagation pros and cons

Is Backpropagation a Good Learning Method?

Advantages

- Widely used
- Effective for multilayer networks
- Works well in practice

Limitations

- Not biologically realistic
- Biological neurons do not propagate errors backward
- May converge slowly
- Can get stuck in local minima

Another Limitation of Backpropagation

Computational Complexity

Backpropagation involves:

- Forward propagation
- Error computation
- Backward propagation
- Weight updates

for every training sample.

Result

Calculations become extensive, and training can be slow — especially for large networks.

Reality in Applications

A pure backpropagation algorithm is rarely used in modern practical systems.

- Slow convergence
- Sensitive to learning rate
- May oscillate or stagnate

Engineering vs Biology

Backpropagation is an engineering optimization method, not a biological learning mechanism.

Takeaway

Despite its limitations, backpropagation remains one of the most powerful and practical training algorithms in machine learning.

THANKS